

Neuro-Evolved Heuristics for Variable Gapped Common Subsequence Identification

Marko Djukanović^{1, 5} Christian Blum² Aleksandar Kartelj³ Sašo
Džeroski⁴ Žiga Zebec⁵

¹University of Nova Gorica, Nova Gorica, Slovenia

²Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain

³Faculty of Mathematics, University of Belgrade, Belgrade, Serbia

⁴Jožef Stefan Institute, Ljubljana, Slovenia

⁵Institute of Information Sciences (IZUM), Maribor, Slovenia

– OPTIMA 2026: XVII International Conference Optimization and Applications,
Petrovac, Montenegro –



Sofinancira
Evropska unija



Introduction

- Objects we deal with: sequences (strings) over finite alphabet Σ
 - DNA/RNA sequences over $\{A, T, G, C/U\}$
 - Proteins over 20 (canonical) amino acids: $\{A, C, D, E, P, Q \dots\}$
- One of central tasks in computational biology:
 - sequence comparison, finding **common motifs** between sequences
 - how structural similarity affects sequence functionality
- Subsequences: **Longest common subsequence problem** variants (studied 50 years already)

Longest common subsequence problem (LCSP)

Definition (LCSP)

Input: Given an arbitrary set of sequences $S = \{s_1, \dots, s_m\}$

Task: Find a subsequence s **common** for all sequences from S with **maximum** possible length ($|s|$).

Example

Input: $S = \{\text{AATTGC}, \text{ATTAC}\}$

LCS solution: $s = \text{ATTC}$

LCS: Literature & Problem Variants

- For $m = 2$, polynomially solvable: **Dynamic programming**
- When m arbitrary large – $\mathcal{NP}$ -hard:
 - approximation, meta-heuristic, but also exact approaches

Problem-related practical variants:

- Repetition-free, filled LCS problem, **Gapped LCS problem**

The gapped LCS problem (Peng and Yang, 2012)

Definition (A gap sequence)

Given is a sequence s and an assigned function $G_s: \{1, \dots, |s|\} \mapsto \mathbb{N}$. A pair (s, G_s) is called a gap sequence.

(To each position of seq. s , one number assigned)

Definition (A gapped subsequence)

Sequence $\tilde{s}$ is a **gapped subsequence** of (s, G_s) iff

- $\tilde{s}$ is a **subsequence** of s
- the gap constraint G_s fulfilled resp. *positions of embedding* $\tilde{s}$ in s :
 - suppose $i_1 < \dots < i_{|\tilde{s}|}$ are those **positions**
 - $(\forall j = 2, \dots, |\tilde{s}|) \ i_j - i_{j-1} \leq G_s(i_j) + 1$ (consecutive positions respect gap distances = allowed *number of letters inbetween*)

Problem definition

Example

$s = \text{AATTGC}$, $G_s(\cdot) = 1$

- $\tilde{s} = \text{ATG}$, the embedding: **AATTGC** (valid gapped subsequence)
- $\tilde{s} = \text{ATG}$, the embedding: **AATTGC** (invalid gapped subsequence)

Definition (The multiple (variable) gapped LCS problem – MVGLCSP)

Input: Given is a set of gapped sequences $\{(s_1, G_{s_1}), \dots, (s_m, G_{s_m})\}$.

Task: Find the longest sequence $\tilde{s}$ so that

- $\tilde{s}$ is **common subsequence** of each s_i
- $\tilde{s}$ **fulfills all gap constraints** G_{s_i} ($i = 1, \dots, m$)

Note: When $G_{s_i} = n$ (length of longest sequence) $\Rightarrow$ VGLCSP = LCSP.

Literature & Motivation for VGLCSP

- Peng and Yang (2012, 2014): studied $m = 2$ version by **three dynamic programming (DP)** approaches
- Manea et al. (2024): complexity analysis investigated
 - **NP-hard** under arbitrary large m
- Djukanovic et al. (2026): the first approach for general VGLCSP (m arbitrary)

Motivation:

- **Genetics and molecular biology**: DNA/protein comparison analysis where structural distances between residues must be respected
- **Time-series analysis**: in settings where events are required to occur within specified temporal delays (Lainscsek et al. (2015))

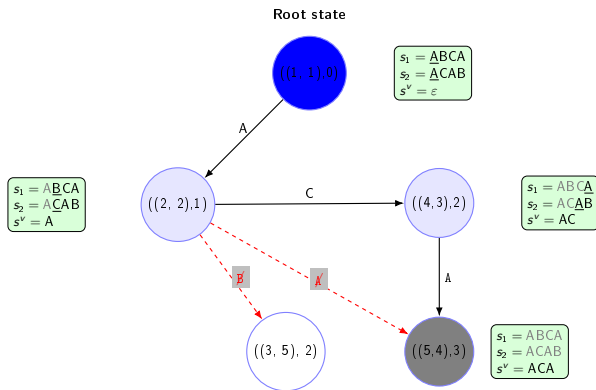
Methodology by Djukanovic et al. (2026)

- Based on the **state space graph formulation** of the problem
- **Nodes** corresp. to partial solutions and resp. subproblems be solved
- **Gap constraints**: incorporated to cut-off invalid extensions (edges) among the LCS extensions immediately
- **Many root/source nodes** in the state graph—the position of leading letter in each common gapped subsequence free to select
 - root state space subgraphs (formulation) w.r.t. starting point (match)

Root-based state graph formulation: example

Given $s_1 = \boxed{A}BCA$, $s_2 = \boxed{A}CAB$, assuming $G_1 = G_2 = 1$, match $p^L = (1, 1)$;

SubSpace((1, 1)): **partial solutions start with A at (1, 1)**



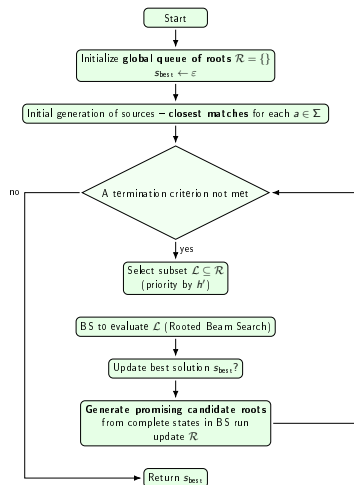
SubSpace((1, 1)) $\implies$ ACA

An issue with the root-state-space formulation: disconnected components

- Generation of longest solution for a root (sub-)space is generally hard
⇒ need an approx. approach (beam search – limited BFS)
- The root-state-space formulation may involve disconnected components generation ⇒ needs iterative beam search to explore all space

⇒ Iterative Multi Source Beam Search (IMSBS) approach.

Workflow of the Iterative Multi-source Beam Search – IMSBS



Heuristic guidances in IMSBS

The best literature heuristic to guide BS and prioritize root node:

- ① UB₂: *Character Frequency Alignment* score the best performing

Experimental evaluation & Conclusions:

- IMSBS able to efficiently run on large synthetic instances
- **Weaknesses:** the performance of UB₂ seems to significantly drop for large n ($n = 500$) and small $|\Sigma|$ ($|\Sigma| = 2$)

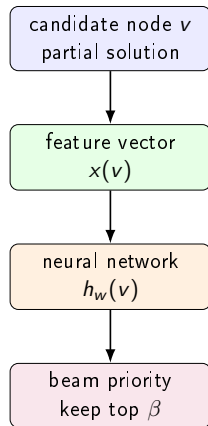
⇒ **Construction of a new heuristic guidance**

Designing advanced heuristic guidances: main idea

- IMSBS needs a score to decide which candidate nodes stay in the beam
- Hand-crafted upper bounds useful, but less reliable for hard settings
- We learn a **node evaluator**: a function assigning a promise score to a partial VGLCS solution

Role of the learned heuristic

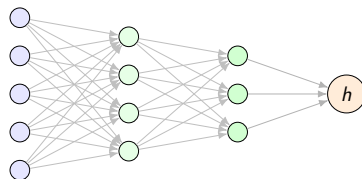
It does **not** construct the solution directly; it ranks candidate states during search.



Neural network basics: MLP as a scoring function

- We use a feed-forward multilayer perceptron (MLP).
- Input: **compact numerical description** of the state v .
- Hidden layers transform features into nonlinear combinations.
- Output: a **single heuristic score** $h_w(v)$.

features hidden + activation funcs score



$$a^{(l+1)} = \phi(W^{(l)}a^{(l)} + b^{(l)}), \quad h_w(v) = a^{(L)}.$$

State features for VGLCS node evaluation

For a state $v = (p^v, l^v)$, we build a size-independent feature representation.

Search progress

- normalize positions:
 $F_i^v = p_i^v / |s_i|$
- max, min, mean, std of F^v
- current length l^v

Gap flexibility

- $G^{\min}, G^{\max}$
- $G^{\text{avg}}, G^{\text{std}}$
- describes how restrictive future extensions are

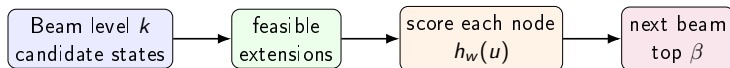
Global context

- alphabet size $|\Sigma|$
- number of sequences m
- standardized before training

Feature sets tested: $F_1 = \text{progress} + \text{gaps} + \text{length} \text{ (9)}$; $F_2 = F_1 + |\Sigma| \text{ (10)}$;
 $F_3 = F_2 + m \text{ (11)}$.

How the learned evaluator is used inside IMSBS

- 1 Beam search expands the currently selected states.
- 2 Each feasible extension produces a child node u .
- 3 The child is transformed into features $x(u)$.
- 4 The NN score $h_w(u)$ is computed (feed-forward propagation).
- 5 Candidates are sorted and only the best β nodes survive.



Important distinction

Learning is **offline**; during the final run the NN is only a fast scoring function.

Why not standard supervised backpropagation?

Obstacle

- No direct target labels for “good node” on large VGLCSP instances
- Search quality non-differentiable: one ranking decision affect many later states
- Exact optimal trajectories unavailable for the general case

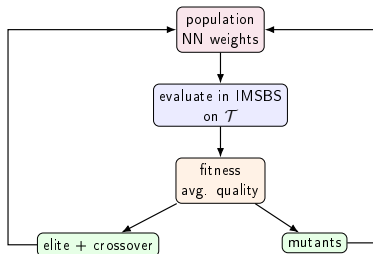
Solution: neuro-evolution

- Treat all NN weights as **one vector** w .
- Evaluate w by running IMSBS on training instances.
- Fitness = average subsequence length found.

$$\max_w \frac{1}{|\mathcal{T}|} \sum_{I \in \mathcal{T}} f(\text{IMSBS}(I, h_w)).$$

Learning weights with BRKGA

- Individual = complete NN weight vector w .
- Initial weights sampled from $[-1, 1]$.
- Population refined by elitism, mutants and biased crossover.
- Validation set monitors generalization and prevents overfitting.



Parameters used

$n_{pop} = 20$, $n_{elite} = 1$, $n_{mutants} = 7$,
 $p_{bias} = 0.5$.

Rank-based ensemble: learned + analytical guidance

- Preliminary tests: the learned heuristic strong, but not uniformly best
- Hand-crafted upper bounds useful, especially in some small- m cases
- Raw scores not directly comparable, **rankings** rather than scores combined

Combined rank

$$r(v) = \alpha r_{h_w^*}(v) + (1 - \alpha) r_{UB}(v).$$

Lower rank $r(v)$ means higher priority in the beam.

m	2	3	5	10
α	0.60	0.50	0.35	0.15

What was tuned? Experimental design

IMSBS parameters

- tuned by irace (parameters from the literature)

Training protocol

- IMSBS utilized $\beta = 500, 100$ beam iterations, max. 60 sec

NN design choices

- 2 or 3 hidden layers
- 5 or 10 neurons per hidden layer
- activation: tanh, ReLU, sigmoid
- features: F_1, F_2, F_3

Selected architectures

m	layers	act.	features
2	10-5-5	ReLU	F_2
3	10-10-5	ReLU	F_2
5	10-5-5	sigmoid	F_1
10	10-5-5	ReLU	F_2

Numerical results: Random and Real benchmarks

Random benchmark

comparison [Ensemble vs. IMSBS]	wins	ties	losses
all groups (32)	20	8	4
$m = 2, 3$	10	5	1
$m = 5, 10$	10	3	3

Wilcoxon: $p = 0.009$ on all groups.

Real benchmark

comparison	wins	ties	losses
LIMSBS-Ensemble vs. IMSBS (20)	12	7	1

Comparable runtime; larger gains on harder instances such as Virus.

Takeaway

The learned ensemble improves solution quality consistently, while keeping the original IMSBS machinery almost unchanged.

Comparison for $m = 2$: gap to exact DP-1 results

Instance ($m, n, \Sigma $)	DP-1	Ensemble	gap
(2, 50, 2)	38.1	37.7	1.0%
(2, 50, 4)	30.3	30.1	0.7%
(2, 100, 2)	77.4	72.4	6.5%
(2, 100, 4)	62.3	62.1	0.3%
(2, 200, 2)	156.4	140.7	10.0%
(2, 200, 4)	127.2	125.8	1.1%
(2, 500, 2)	395.9	299.5	24.4%
(2, 500, 4)	317.2	310.9	2.0%

Observation

For $|\Sigma| = 4$, the ensemble is close to DP-1. The most difficult case are binary alphabets with long sequences, especially (2, 500, 2).

Conclusion

- Learning an improved guidance promising approach
- Low alphabet diversity, despite improved, still requires stronger exploration/richer features.